

[운영체제_실습3_2018320184_강민규]

01

process_01.c

코드 완성

</01>	default
</02>	case 0
</03>	getpid()
</04>	0

출력 결과

```
process_01.c
kmg@2018320184:~/shared_folder/process$ gcc -o process_01 process_01.c
kmg@2018320184:~/shared_folder/process$ ./process_01
[1096] (Parent) I am old and calm.
[1097] (Child) I am young and wild.
kmg@2018320184:~/shared_folder/process$ _
```

02

process_02.c

코드 완성

</01>	pid > 0
</02>	pid == 0
</03>	0
</04>	val

출력 결과

```
process_01 process_02 process_03
kmg@2018320184:~/shared_folder/process$ gcc -o process_02 process_02.c
kmg@2018320184:~/shared_folder/process$ ./process_02
The original value is 10
The parent will now add 1 and the child will subtract 3
The value of parent is 11.
kmg@2018320184:~/shared_folder/process$ The value of child is 7.
```

03

process_03.c

코드 완성

</01>	"/bin/pwd", "pwd", NULL
-------	-------------------------

출력 결과

```
process_01 process_01.c process_02 process_02.c process_03 process_03.c
kmg@2018320184:~/shared_folder/process$ gcc -o process_03 process_03.c
kmg@2018320184:~/shared_folder/process$ ./process_03
where am I?
/home/kmg/shared_folder/process
kmg@2018320184:~/shared_folder/process$
```

04

process_04.c

코드 완성

</01>	default
</02>	case 0
</03>	pid2
</04>	child_name, name, getpid()
</05>	case 0
</06>	grandchild_name, child_name, name, getpid()

출력 결과

```
process_01 process_01.c process_02 process_02.c process_03 process_03.c process_04 process_04.c
kmg@2018320184:~/shared_folder/process$ gcc -o process_04 process_04.c
kmg@2018320184:~/shared_folder/process$ ./process_04
My name is Jeffrey and I am a parent.
My name is Michael and my father is Jeffrey. My pid is 1205
My name is Steven! My father's name is Michael and my grandpa is called Jeffrey. My pid is 1206
kmg@2018320184:~/shared_folder/process$
```

05

process_05.c

코드 완성

</01>	pid > 0
</02>	&status
</03>	status / 256
</04>	pid == 0
</05>	3

출력 결과

```
kmg@2018320184:~/shared_folder/process$ gcc -o process_05 process_05.c
kmg@2018320184:~/shared_folder/process$ ./process_05
Counting to three.
1!
2!
3!
kmg@2018320184:~/shared_folder/process$
```

06

process_06.c

코드 완성

</01>	pid > 0
</02>	pid2 > 0
</03>	pid, &status, 0
</04>	pid2, &status, 0
</05>	pid2 == 0
</06>	pid == 0

출력 결과

```
kmg@2018320184:~/shared_folder/process$ gcc -o process_06 process_06.c
kmg@2018320184:~/shared_folder/process$ ./process_06
counting to 5!
1!
2!
3!
4!
5!
kmg@2018320184:~/shared_folder/process$
```

07

process_07.c

코드

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  #include <stdlib.h>
6  #define SLEEP_TIME 2
7
8  int main(int argc, char* argv[]) {
9      pid_t pid;
10     int status;
11     int parent_pid, child_pid, favorite_number;
12     char favorite_fruit[] = "Apple";
13
14     printf("Final Question!\n");
15
16     pid = fork();
17
18     switch (pid) {
19     case -1:
20         printf("fork failed\n");
21         return -1;
22
23     default:
24         // hint: Which case handles the parent process based on the return value of fork?
25         // hint: How can the parent process retrieve its own unique PID?
26         parent_pid = getpid();
27         printf("[%d] I am a parent\n", parent_pid);
28
29         // hint: How can the parent block its execution until the child finishes?
30         wait(&status);
31
32         printf("[%d] ....and my child's favorite number is %d\n", parent_pid, WEXITSTATUS(status));
33         break;
34
35     case 0:
36         // hint: Which case handles the child process?
37         // hint: To ensure the parent prints its message first, what should the child do?
38         sleep(SLEEP_TIME);
39
40         favorite_number = 5;
41
42         // hint: How can the child identify its own PID?
43         child_pid = getpid();
44
45         // hint: How can the child identify its parent's PID?
46         parent_pid = getppid();
47
48         printf("[%d] and I am a child! And my parent's PID is [%d]\n", child_pid, parent_pid);
49         printf("[%d] my parent's favorite fruit is %s but he doesn't know that my favorite number is %d\n", child_pid, favorite_fruit, favorite_number);
50
51         // hint: How does the child terminate and pass its favorite number to the parent?
52         exit(favorite_number);
53         break;
54     }
55
56     return 0;
57 }
```

Line23~34(default case): -1, 0 이외의 양수들에 대해서는 default로 처리할 수 있다.
이는 parent process를 처리하는 case가 된다.

Line26[parent_pid = getpid();]: getpid() 함수를 호출하여 현재 코드를 실행 중인 parent process 자신의 PID를 가져와 parent_pid에 저장한다.

Line30[wait(&status);]: child process 가 종료될 때까지 parent process 의 실행을 일시적으로 멈추고 대기하게 만드는 wait() system call 을 사용했다. child 가 종료되면서 넘겨주는 상태 값을 받아오기 위해 status 변수의 주소(&status)을 인자로 넣었다.

Line32[printf("[%d]and my child's favorite number is %d\\n", parent_pid, WEXITSTATUS(status));]: child process 가 exit()을 통해 전달한 값을 출력하기 위해 WEXITSTATUS(status) 매크로를 사용했다. child 가 넘겨준 상태 값은 시스템을 거치며 256 이 곱해진, 즉 8 비트만큼의 이동이 발생한 채로 status 에 저장되므로, 이 매크로를 통해 원래의 순수한 종료 값(5)을 출력한다.

Line35~54(case 0): 생성된 child process 는 pid 에 0 이 할당된다.

Line38[sleep(SLEEP_TIME);]: child process 의 실행을 매크로로 정의된 SLEEP_TIME 으로 일시 정지시킨다. 이를 통해 parent process 가 child 보다 먼저 첫 문장을 화면에 출력할 수 있도록 실행 순서를 제어한 것이다.

Line43[child_pid = getpid();]: child process 자신의 고유 PID 를 가져오기 위해 getpid() 함수를 호출하고 이를 child_pid 변수에 저장한다.

Line46[parent_pid = getppid();]: 현재 자신을 생성한 parent process 의 PID 를 알아내기 위해 getppid() 함수를 호출하여 parent_pid 변수에 덮어쓴다.

Line48[printf("[%d] and I am a child! And my parent's PID is [%d]\\n", child_pid, parent_pid);]: 앞서 getpid()와 getppid()를 통해 획득한 child process 의 PID(child_pid)와 parent process 의 PID(parent_pid)를 지정된 양식에 맞춰 화면에 출력시킨다.

Line49[printf("[%d] my parent's favorite fruit is %s but he doesn't know that my favorite number is %d\\n", child_pid, favorite_fruit, favorite_number);]: child process 의 PID(child_pid)와 더불어, parent 로부터 메모리 공간을 공유(Copy-on-Write)받아 그대로 사용할 수 있는 favorite_fruit 문자열, 그리고 child process 내부에서 5 로 지정한 favorite_number 정수 값을 순서대로 출력하여 예상 결과값과 일치시킨다.

Line52[exit(favorite_number);]: child process 가 자신의 모든 작업을 마치고 완전히 종료하도록 exit() system call 을 사용했다. 이때 parent 에게 전달할 값으로 favorite_number 를 인자로 넣어, parent 의 wait() 함수를 통해 이 값이 성공적으로 전달되도록 한다.

출력 결과

```
kmg@2018320184:~/shared_folder/process$ gcc -o process_07 process_07.c
kmg@2018320184:~/shared_folder/process$ ./process_07
Final Question!
[1078] I am a parent
[1079] and I am a child! And my parent's PID is [1078]
[1079] my parent's favorite fruit is Apple but he doesn't know that my favorite number is 5
[1078] ....and my child's favorite number is 5
kmg@2018320184:~/shared_folder/process$
```